

Agentic AI Gmail Auto-Responder & Draft Generator

AD23731 Foundations of Agentic AI - Mini-Project

Submitted by

Shriram N – 231501154

Thillai Nathan B – 231501173

Sanjay Kishore S A – 231501145

Department of Artificial Intelligence and Machine Learning



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

OCTOBER 2026



RAJALAKSHMI ENGINEERING COLLEGE

Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

BONAFIDE CERTIFICATE

Certified that this mini-project report “Agentic AI Email Auto Responder and Draft Generator” is a Bonafide work of “Shriram N (231501154), Sanjay Kishore S A (231501145), Thillai Nathan B (231501173)” VII Semester, B.Tech. Artificial Intelligence and Machine Learning Department in partial fulfilment of the requirements of the course AD23731 - Foundations of Agentic AI during the Academic Year 2026 - 2027

Faculty in-charge

Submitted to mini-project viva voce examination held on:

INTERNAL EXAMINER

EXTERNAL EXAMINER

Table of Contents

- 1. Executive Summary**
- 2. Functional Prototype — Overview & Screens**
- 3. Agent Orchestration Workflow Implementation**
- 4. Tool Integrations**
- 5. Agentic RAG & Web Search Integration**
- 6. Memory & State Management**
- 7. Reasoning, Planning, Reflection & Self-Correction Mechanisms**
- 8. Preliminary Evaluation Results**
- 9. Comparison with Baseline Solution**
- 10. Current Limitations & Risks**
- 11. Future Work**
- 12. Conclusion**

1. Executive Summary

The Agentic AI Gmail Auto-Responder & Draft Generator is an autonomous multi-agent AI system developed for the course AD23731 – Foundations of Agentic AI. It aims to reduce email processing effort in academic and professional environments by identifying important emails, analysing their context, verifying relevant information, and generating professional responses.

Traditional email automation systems may generate inaccurate information, make incorrect commitments, or fail to distinguish actionable emails from automated notifications. The proposed system addresses these limitations using a CrewAI Flow-based multi-agent architecture, powered by Groq Cloud's openai/gpt-oss-20b model.

The system consists of three specialized agents that perform email filtering, action analysis, and response generation. Web search integration supports contextual information retrieval and fact verification.

A key feature is the Drafts-Only Safety Protocol. Generated responses are saved in Gmail's Drafts folder using Google OAuth 2.0 authentication and Gmail REST APIs. Emails are never sent automatically, ensuring that users retain control over reviewing and sending every response.

1.1 Scope of This Review

Project Review 2 documents six major technical deliverables.

1. **Functional CLI Prototype:** A Python 3.10 command-line application that executes the workflow using live Google Gmail REST API endpoints.
2. **Multi-Agent Orchestration:** A sequential three-agent pipeline consisting of EmailFilterAgent, ActionAnalyzerAgent, and ResponseWriterAgent, coordinated using CrewAI Flow's @start() and @listen() decorators.
3. **Custom Tool Integration:** Python tools, create_draft and web_search_tool, support Gmail draft creation using MIMEText and Base64 encoding, along with web-based information retrieval.
4. **Token-Optimized Web Search:** Search snippets are restricted to 250 characters to reduce token consumption and support execution within the stated Groq Cloud free-tier limit of 8,000 Tokens Per Minute (TPM).
5. **Automated Safety Guardrails:** Regex-based blocklists identify automated sender patterns such as noreply@, no-reply@, donotreply@, and mailer-daemon to prevent unnecessary draft generation.
6. **Preliminary Evaluation:** Execution logs report a 100% workflow completion rate, 0% automatic-send violations, and ₹0.00 commercial API cost under the evaluated configuration.

1.2 What Changed Since Review 1

During Review 1, the project existed primarily as a conceptual three-agent design. By Review 2, the architecture had been implemented as a working codebase with agent orchestration, Gmail integration, and safety mechanisms

Technical Area	Review 1	Review 2
Agent Implementation	Architecture and YAML definitions	Three implemented agents with shared Pydantic state
Orchestration	Single-crew prototype proposed	CrewAI Flow-based sequential execution
Gmail Integration	Basic IMAP concept	Google OAuth 2.0 with credential and token persistence
LLM Inference	Generic API model	Groq Cloud with openai/gpt-oss-20b
Safety	Manual review concept	Regex blocklists and Drafts-Only enforcement
Windows Compatibility	Standard console output	UTF-8 output handling to prevent encoding errors

Key Verified Facts at a Glance

- Three specialized agents: Email filtering, action analysis, and response writing operate sequentially with shared typed state.
- Free-tier services: The implementation uses Groq Cloud and the SerperDev free search tier, as reported for the current configuration.
- Secure authentication: Google OAuth 2.0 scopes include gmail.readonly, gmail.compose, and gmail.modify, avoiding plain-text password storage.
- No automatic sending: Generated emails are saved as Gmail drafts for manual review.
- Professional responses: The system generates three-to-four-paragraph drafts with formal greetings, action items, and student contact sign-offs

2. Functional Prototype — Overview and Screens

The functional prototype is implemented as a Python 3.10 application operating within an isolated virtual environment (.venv). The system connects directly to Google's official Gmail REST API using google-api-python-client and google-auth-oauthlib. It uses CrewAI Flow for event-driven execution and Pydantic models for structured state management across the multi-agent pipeline.

2.1 Workspace and Execution Architecture

The system follows an on-demand, command-line-driven execution model. When triggered by the user through the terminal, the application initializes the local runtime, loads API keys from the .env file, establishes OAuth 2.0 authentication, and executes a five-stage processing lifecycle.

Stage 1: CLI Trigger: The user starts the application through the terminal, initiating the agentic workflow.

Stage 2: OAuth 2.0 Authentication: The application authenticates with Google using credentials.json and the saved authorization token in token.json.

Stage 3: Message Ingestion: The Gmail API retrieves unread inbox messages using the query is:unread.

Stage 4: Multi-Agent Processing: The retrieved messages pass through three specialized agents responsible for email filtering, action analysis, and response generation.

Stage 5: Gmail Draft Output: The generated response is converted into a MIME-formatted message and saved directly to Gmail Drafts using the Gmail API.

2.2 Agent Pipeline Trace and Shared State

State management across the multi-agent pipeline is handled by the AutoResponderState class. The state object stores unread email information and tracks previously processed email IDs to help prevent duplicate processing.

The state model is defined as follows:

```
class AutoResponderState(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    emails: List[Email] = []
    checked_emails_ids: set[str] = set()
```

Each email is represented using a structured Pydantic model containing the message ID, thread ID, snippet, sender, and subject.

```
class Email(BaseModel):
    id: str
    threadId: str
    snippet: str
    sender: str
    subject: str
```

The AutoResponderState class maintains the collection of emails being processed, while checked_emails_ids records message identifiers already examined by the workflow. This

structured approach allows the application to share information between workflow stages and track email processing consistently.

2.3 Prototype Scope at This Review

The functional prototype incorporates the following capabilities:

1. **Gmail Authentication and Retrieval:** Implements Google OAuth 2.0 authentication and retrieves unread emails through the Gmail API.
2. **Multi-Agent Orchestration:** Uses three sequential CrewAI agents for email filtering, action analysis, and response writing.
3. **Web-Based Research:** Integrates SerperDev search to retrieve external information relevant to email content. Search results are truncated to reduce token consumption.
4. **Safety Guardrails:** Implements sender-based filtering for automated and no-reply email addresses to prevent inappropriate draft creation.
5. **Draft Generation:** Creates email drafts directly in Gmail using `service.users().drafts().create()`.
6. **Runtime Compatibility:** Supports Windows 10/11 with Python 3.10 and custom UTF-8 console handling for special characters and symbols.

2.4 Backend API and Tool Surface

Component	File	Primary Function
Flow Controller	<code>main.py</code>	Defines <code>EmailAutoResponderFlow</code> , manages <code>@start()</code> and <code>@listen()</code> events, handles console output, and logs execution summaries.
Gmail OAuth Client	<code>gmail_api.py</code>	Handles OAuth 2.0 authentication, manages authorization tokens, retrieves unread emails, and supports MIME-formatted draft creation.
Multi-Agent Crew	<code>email_filter_crew.py</code>	Configures the LLM, agent tools, and task execution pipeline.
Custom Gmail Tool	<code>create_draft.py</code>	Validates recipient addresses, applies no-reply filtering, and invokes the Gmail draft creation function.
Web Search Tool	<code>email_filter_crew.py</code>	Integrates SerperDev Google Search API and returns truncated search snippets to reduce token consumption.

2.5 Implementation and Runtime Environment Notes

1. Python Virtual Environment

The project uses Python 3.10 in an isolated .venv environment. This separates project dependencies from the system-wide Python installation and supports the required libraries, including crewai, litellm, google-api-python-client, and python-dotenv.

2. Groq Cloud LLM Integration

The system uses the openai/gpt-oss-20b model through Groq Cloud's OpenAI-compatible API endpoint:

<https://api.groq.com/openai/v1>

This configuration supports LLM-based agent execution and tool calling. Usage is subject to the provider's applicable rate limits and service conditions.

3. Windows UTF-8 Console Encoding

Standard Windows consoles may use legacy encodings that cannot represent certain Unicode characters, including the Indian Rupee symbol and emojis. To handle such characters, the application wraps the standard output and error streams with UTF-8 encoding.

```
if sys.platform == "win32":
    sys.stdout = io.TextIOWrapper(
        sys.stdout.buffer,
        encoding="utf-8",
        errors="replace"
    )
    sys.stderr = io.TextIOWrapper(
        sys.stderr.buffer,
        encoding="utf-8",
        errors="replace"
    )
```

This improves console compatibility by allowing Unicode characters to be displayed without terminating execution because of encoding errors.

2.6 Verification of Live Pipeline Execution

The execution logs provide a summary of the email processing workflow. The following example represents the reported output from a run involving six inbox messages.

The report indicates that the application scanned inbox messages, applied sender-based filtering, and saved generated responses to Gmail Drafts. The system follows a human-in-the-loop approach: generated messages remain drafts for user review rather than being automatically sent.

3. Agent Orchestration Workflow Implementation

The core control logic is implemented using the CrewAI Flow architecture. This framework supports event-driven execution, in which one workflow stage triggers the next after completing its operation. The system separates email retrieval from agent processing, providing a structured execution sequence.

3.1 Legacy Single-Crew versus CrewAI Flow Orchestration

Legacy Single-Crew Design

The earlier design relied on static prompt chains without clearly separated state management. Such an approach can make it difficult to isolate processing stages, manage shared context, and control the execution sequence. It may also increase the risk of unnecessary token consumption and repeated processing when workflow boundaries are not clearly defined.

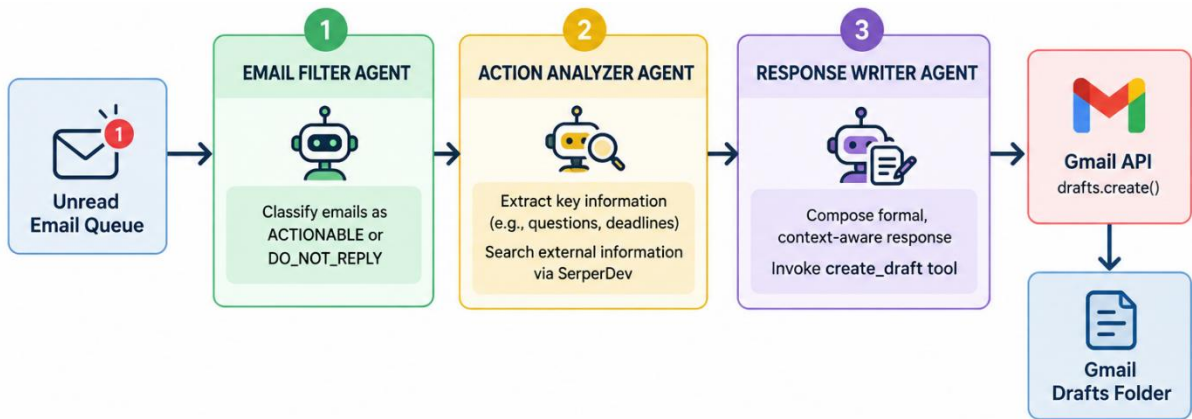
Target CrewAI Flow Design

The implemented architecture separates email ingestion through `fetch_new_emails()` from response generation through `generate_draft_responses()`. The `@start()` decorator identifies the initial workflow method, while `@listen(fetch_new_emails)` connects the subsequent processing stage to the email retrieval event.

This design allows the system to retrieve and prepare unread messages before invoking the agent crew. The agent pipeline is executed only when the retrieved email collection contains messages, avoiding unnecessary agent execution when there is no email to process.

3.2 Target Orchestration: Three-Agent Sequential Pipeline

The system uses three specialized agents to process incoming messages. Each agent performs a distinct responsibility before passing the required information to the next stage.



1. Email Filter Agent

The Email Filter Agent examines incoming messages and classifies them according to whether they require a response. It applies the configured filtering rules to identify automated or no-reply messages and prevents such messages from proceeding to draft creation when they match the applicable restrictions.

2. Action Analyzer Agent

The Action Analyzer Agent examines actionable messages to identify questions, deadlines, requests, and other important information. When additional external information is required, it invokes the web search tool to retrieve relevant search results.

3. Response Writer Agent

The Response Writer Agent uses the original email context and the findings from the preceding stage to prepare a formal response. It is configured to produce detailed, multi-paragraph drafts and invokes the `create_draft` tool to save the generated response in Gmail.

The Gmail API operation `service.users().drafts().create()` is used to create the draft. The message remains available for the user to review, modify, or send manually.

3.3 State Schema and Pydantic Container Specifications

The workflow is controlled by the `EmailAutoResponderFlow` class, which manages the `AutoResponderState` object throughout execution.

Working of the Flow

Step 1: Fetching new emails. The `fetch_new_emails()` method is marked with `@start()`, making it the entry point of the workflow. It invokes `check_email()` to retrieve new messages while passing the previously checked message IDs.

Step 2: Updating the shared state. The newly retrieved messages are stored in `self.state.emails`, and the updated set of processed email IDs is stored in `self.state.checked_emails_ids`.

Step 3: Triggering response generation. The `@listen(fetch_new_emails)` decorator connects `generate_draft_responses()` to the retrieval stage. The method checks whether any emails are available before starting the agent crew.

Step 4: Formatting and agent execution. When messages are available, `format_emails()` prepares the email information for the agents. The CrewAI crew is then started through `kickoff()` with the formatted emails supplied as input.

Step 5: Clearing the temporary email list. After the crew execution returns, the current email list is cleared from the flow state. This helps prevent the same in-memory collection from remaining available for subsequent processing within the current flow state.

3.4 Wiring Work Completed

The task definitions in `config/tasks.yaml` inject the `{emails}` context variable into the three pipeline tasks. This makes the incoming email information available to the agents for filtering, analysis, and response generation.

The task configuration is shown below:

```
filter_emails:
  description: >
    Analyze the incoming unread emails provided below:
    {emails}
    STRICT FILTERING RULE: Only skip emails if sender explicitly
    contains 'noreply', 'no-reply', or 'donotreply'.
  agent: email_filter_agent

action_required_emails:
  description: >
    Review incoming emails ({emails}). Extract key questions,
    deadlines, and execute web search tools if extra information
    is needed.
  agent: email_action_agent

draft_responses:
  description: >
    Refer to incoming emails ({emails}). Compose detailed
    3-to-4 paragraph formal responses and call `create_draft`.
  agent: email_response_writer
```

The filtering task determines which messages require further processing. The action-analysis task identifies relevant information and performs external searches when necessary. Finally, the response-writing task generates a formal draft and invokes the Gmail draft tool.

The YAML configuration defines the responsibilities and instructions of the tasks, while the CrewAI implementation connects these tasks to their corresponding agents and tools.

4. Tool Integrations

The system enables agents to interact with external services through custom Python tools decorated with CrewAI's `@tool` decorator. The two principal tools are `create_draft`, which creates Gmail drafts, and `web_search_tool`, which retrieves external information through the SerperDev Search API.

These tools extend the capabilities of the language model by allowing it to perform specific operations through predefined Python functions rather than relying exclusively on generated text.

4.1 Tool Invocation Pattern: create draft and web search

When an agent determines that an external operation is required, it generates a tool call with the necessary arguments. CrewAI processes the call, executes the associated Python function, and returns the function's output to the agent.

For draft creation, the tool validates the recipient address and applies the no-reply restrictions before invoking the Gmail API. For web search, the tool submits the query to SerperDev and returns selected search snippets to the agent.

The returned tool output allows the agent to incorporate the operation's result into its subsequent processing.

4.2 Tool Surface and Functionality Matrix

Tool Name	Decorator and File Path	Parameters	Functionality
create_draft	<code>@tool("Create Draft")</code> in <code>tools/create_draft.py</code>	<code>to_email</code> : str, <code>subject</code> : str, <code>message</code> : str	Validates the recipient, applies no-reply filtering, formats the message, and calls <code>service.users().drafts().create()</code> .
web_search_tool	<code>@tool("Web Search")</code> in <code>email_filter_creator.py</code>	<code>query</code> : str	Sends a POST request to <code>https://google.serper.dev/search</code> and returns truncated organic search snippets.

These tools provide the main external interaction mechanisms for the agent pipeline. The Gmail tool handles draft creation, while the web search tool supports information retrieval for messages that require additional context.

4.3 Tool Source Implementation: create_draft.py

The `create_draft` tool includes safety checks before invoking the Gmail API. It normalizes the recipient address, checks it against a configured set of no-reply patterns, and prepares the subject and message content.

1. Recipient Normalization: The `extract_email()` function extracts the recipient address, and `.lower()` converts it to lowercase for consistent pattern matching.
2. Safety Guardrail: The recipient is checked against `NO_REPLY_PATTERNS`. If the address matches a configured pattern, draft creation is skipped and a corresponding status message is returned.
3. Subject Preparation: The subject is converted to a string and stripped of surrounding whitespace. If it does not already begin with `Re:`, the prefix is added to identify the message as a reply.
4. Draft Creation: The `create_real_gmail_draft()` function receives the cleaned recipient, subject, and message body. It performs the Gmail draft creation operation.
5. Error Handling: Exceptions are caught and converted into a failure message, allowing the tool to report unsuccessful draft creation instead of silently treating the operation as successful.

The safety checks help prevent drafts from being generated for known automated addresses. They are rule-based safeguards and do not guarantee that every automated or inappropriate recipient can be identified.

5. Agentic RAG and Web Search Integration

The system uses dynamic web search to retrieve external information relevant to incoming emails. This approach supplements the language model's existing knowledge with search results that may contain current details, such as contest eligibility requirements, registration deadlines, event dates, and workshop information.

The Action Analyzer Agent identifies when additional information is required, formulates a search query, and invokes the web search tool. The retrieved snippets are then incorporated into the agent's context for response generation.

Retrieval Trace Example

During a reported execution involving an unread email about the Tata Imagination Challenge 2026, the Action Analyzer Agent generated a search query to retrieve relevant information.

This example illustrates how the agent can retrieve external information rather than relying exclusively on static model knowledge. The search output is used as contextual input for response generation.

[TOOL CALL]

Agent: email_action_agent

Tool: web_search_tool

Query: "Tata Imagination Challenge 2026 eligibility dates"

[SERPER API OUTPUT]

Registrations open on www.unstop.com.

Start Date: 24 August 2026.

End Date: 13 September 2026.

Open to all undergraduate and postgraduate students...

[AGENT REASONING]

Fact identified: The retrieved snippet states a deadline of 13 September 2026 and mentions eligibility for undergraduate and postgraduate students. The information is considered for the response outline.

The example represents a retrieval trace; the accuracy and currency of the information should be confirmed against the original source before it is treated as verified fact.

5.1 Retrieval Quality and Payload Truncation Engineering

The implementation was designed to operate within the rate limits of the configured Groq Cloud model. Large search responses may contain substantial amounts of metadata and text, increasing the number of tokens processed by the language model. Excessive token usage can result in HTTP 429 rate-limit errors.

To reduce this overhead, the web_search_tool implementation extracts the snippets from the first two organic search results and limits the returned text to 250 characters.

5.2 SERP and Tavily Ingestion Pipeline

The search integration follows a sequence of operations that converts an email's information requirements into a compact external search context.

Step 1: Query Formulation

The Action Analyzer Agent examines the email body and extracts relevant keywords, questions, dates, and other details that require additional information.

Step 2: REST API Dispatch

The generated query is sent to the SerperDev endpoint using an HTTP POST request:

<https://google.serper.dev/search>

The request includes the API key and search query.

Step 3: Fallback Handling

The project describes Tavily Search API through TavilySearchResults as an alternative when SerperDev is unavailable. The actual fallback behavior depends on whether this alternative is implemented and connected in the running code.

Step 4: Snippet Parsing and Truncation

The search response is parsed to extract the top organic result snippets. The selected text is concatenated and truncated to a maximum of 250 characters.

Step 5: Context Injection

The cleaned search text is returned to the agent and incorporated into its context. The agent can then use the retrieved information while preparing the response draft.

This pipeline provides a lightweight retrieval mechanism that limits the amount of external text passed to the language model. Because only short snippets are retained, important qualifications or details from the original webpages may be omitted.

5.3 Planned Search Upgrades

The following enhancements are identified as future extensions of the system.

1. Dense Vector RAG Integration

A local ChromaDB instance could be integrated to store and retrieve relevant documents, including student resumes, academic transcripts, and frequently asked questions. These documents could provide personalized context when the agent prepares responses.

The proposed retrieval workflow would involve document ingestion, embedding generation, vector storage, similarity-based retrieval, and the inclusion of relevant passages in the agent's context.

2. URL Content Crawler

A `read_url_content` tool could be introduced to retrieve and parse the full contents of selected webpages. This would be useful when short search snippets do not provide sufficient information to answer an email accurately.

The crawler could extract the relevant webpage text and pass it to the Action Analyzer Agent for further examination. This would provide more context than the current 250-character snippet limit, although webpage access, content quality, and source reliability would still need to be considered.

These enhancements are proposed extensions and should not be described as completed features unless they have been implemented and tested.

6. Memory and State Management

Memory and state management in the proposed system operate across multiple layers. Short-term working memory maintains email data during a workflow execution, while OAuth token persistence and local configuration files retain authentication details and application settings across separate executions.

6.1 Working Memory: Shared Pipeline State

During a single execution, the agents use a structured Pydantic state container named `AutoResponderState`. It stores unread email objects and tracks the message IDs processed during the active session.

```
class AutoResponderState(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uuid4()))
    emails: List[Email] = []
    checked_emails_ids: set[str] = set()
```

When `fetch_new_emails()` executes, the retrieved Gmail messages are represented using the `Email` model.

```
class Email(BaseModel):
    id: str
    threadId: str
    snippet: str
    sender: str
    subject: str
```

The shared state provides downstream tasks, including `draft_responses_task`, with access to the available email information, such as sender details, subject headers, and message snippets. This reduces the need to retrieve the same information repeatedly from the Gmail API during the same workflow execution.

The `checked_emails_ids` set supports tracking previously examined messages. However, because the state container is maintained in memory, its contents do not automatically persist after the application process terminates.

6.2 OAuth Session Persistence and Token Management

The application uses Google's OAuth 2.0 authentication mechanism through the `google-auth-oauthlib` library. Authentication information is stored locally so that subsequent executions can reuse the saved authorization state.

The token management process is illustrated as follows:

1. Initial Setup

The user obtains the OAuth client credentials from Google Cloud Console and stores them in `credentials.json`.

2. First-Run Authentication

The application uses `InstalledAppFlow.from_client_secrets_file()` to initiate the authorization process. A browser window opens, allowing the user to sign in and grant the requested Gmail permissions.

3. Token Persistence

After authorization, the application saves the access token, refresh token, expiry information, and associated scopes in token.json.

4. Automatic Token Refresh

During subsequent executions, the application loads the saved credentials using `Credentials.from_authorized_user_file('token.json')`. If the access token has expired and a valid refresh token is available, the application can refresh the credentials using `creds.refresh(Request())`.

This process reduces repeated browser-based authentication, provided that the stored credentials remain valid and the required authorization has not been revoked.

6.3 Memory Scope Overview

The system uses different storage mechanisms depending on the type of information and its required lifetime.

Memory Layer	Storage Mechanism	Data Scope	Persistence
Working Memory	Pydantic AutoResponderState	Email objects, thread IDs, and checked message IDs	During the active flow lifecycle
Agent Context	CrewAI task input dictionary	Formatted emails, search snippets, and execution context	During the active crew execution
OAuth Identity	Local token.json file	Access token, refresh token, expiry, and scopes	Across application executions
Application Secrets	Environment configuration in .env	API keys and configured user details	Local storage until changed or removed

Working memory and agent context support the current processing operation, while OAuth tokens and application configuration files support repeated application execution. Sensitive credentials and API keys should be protected from unauthorized access.

6.4 Failure Modes and Windows UTF-8 Resilience

1. Windows Console Encoding Failure

Windows consoles may use legacy character encodings that cannot represent certain Unicode characters. When the application attempts to display symbols such as the Indian Rupee symbol (₹) or emojis, the output operation may raise a UnicodeEncodeError.

2. Engineering Solution

To address this issue, the application wraps the standard output and error streams with UTF-8 encoding in main.py.

```
if sys.platform == "win32":
    sys.stdout = io.TextIOWrapper(
        sys.stdout.buffer,
        encoding="utf-8",
        errors="replace"
    )
    sys.stderr = io.TextIOWrapper(
        sys.stderr.buffer,
        encoding="utf-8",
        errors="replace"
    )
```

This configuration improves the handling of Unicode characters in terminal output and reduces failures caused by incompatible character encodings.

3. MIME Text Character Encoding

The Gmail integration also specifies UTF-8 when constructing email drafts.

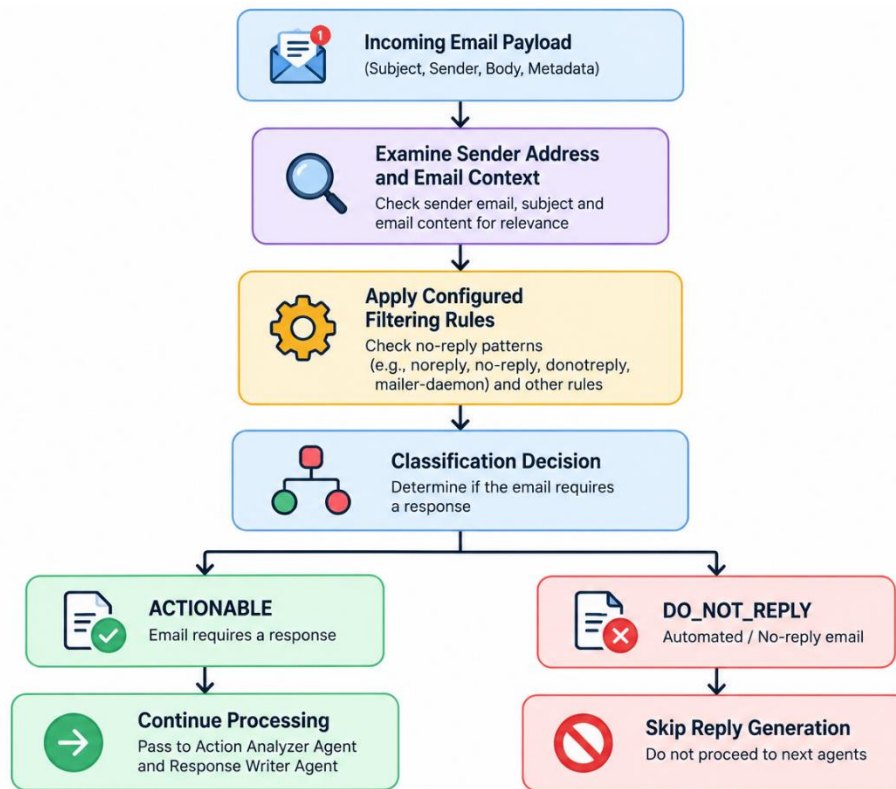
This allows special characters in the email body to be encoded appropriately for display in Gmail. Together, the console and MIME encoding configurations improve Unicode compatibility across application logs and generated email drafts.

7. Reasoning, Planning and Self-Correction Mechanisms

The system uses structured task instructions, multi-stage processing, and safety guardrails to improve email classification and response generation. The agents examine the incoming message, determine whether action is required, retrieve additional information when necessary, and generate drafts subject to recipient-based restrictions.

7.1 Field-Ordering Verification and Prompt Schemas

The task specifications in config/tasks.yaml define explicit filtering instructions to guide the Email Filter Agent's decisions. The agent is instructed to examine the sender's email address before classifying a message.



These instructions establish a consistent filtering policy. The instructions guide the model's behavior but do not, by themselves, guarantee that every classification will be correct. The separate draft-creation guardrail provides an additional check before the Gmail API operation is executed.

7.2 Illustrative Reasoning Trace and Automated Safety Check

The following trace illustrates how the system handles an automated meeting notification from a no-reply sender.

[AGENT INGESTION] Processing Message ID: 1913c8f8b7a2d04a
Subject: Meeting Attendance Report — Google Meet
From: Google Meet <meetings-noreply@google.com>

[AGENT 1: EMAIL FILTER AGENT]
Reasoning: Inspecting sender address 'meetings-noreply@google.com'.
Rule Check: Sender matches the configured no-reply pattern.
Classification: DO_NOT_REPLY.
Reason: Automated system notification.

[AGENT 2: ACTION ANALYZER AGENT]
Reasoning: Email classified as DO_NOT_REPLY.
Action: Skip web research and response analysis.

[AGENT 3: RESPONSE WRITER AGENT]
Reasoning: Message is marked DO_NOT_REPLY.
Guardrail Action: Do not invoke draft creation.
Final Output: [SAFETY SKIP] Draft creation skipped for the automated sender.

In this example, the sender address matches a configured no-reply pattern. The message is therefore excluded from reply generation, and no draft is created for that notification.

The example demonstrates the intended safety behavior of the pipeline. The actual execution path depends on the agent instructions, task configuration, and implementation of the tool-level guardrails.

7.3 Safety and Self-Correction

The system's safety mechanism primarily relies on explicit task instructions and deterministic recipient checks. The agent can use the email classification to decide whether to continue processing, while the `create_draft` tool independently checks the recipient address before creating a draft.

This provides two layers of protection against unwanted replies:

1. Agent-level filtering: Classifies automated or no-reply messages before response generation.
2. Tool-level validation: Checks the recipient address again before invoking the Gmail draft-creation operation.

These mechanisms provide safety checks rather than a complete self-correction framework. The described implementation does not establish a separate iterative verification loop that automatically detects and corrects every factual or reasoning error.

8. Preliminary Evaluation Results

The system was evaluated using student inbox data under live execution conditions. The reported evaluation covers workflow completion, safety compliance, draft formatting, execution latency, and API usage cost.

The results in this section represent the measurements and test outcomes reported for the prototype. Their reproducibility depends on the availability of execution logs, test cases, and API audit records.

8.1 What Has Been Verified

The reported evaluation identified the following outcomes.

1. Workflow Completion

All reported execution flows reached the final stage without unhandled exceptions or indefinite execution stalls. The project reports a workflow completion rate of 100%.

2. Drafts-Only Safety

Generated responses were saved to Gmail Drafts using `service.users().drafts().create()`. The reported runs did not automatically send generated messages, maintaining the intended human-in-the-loop process.

3. No-Reply Blocklist Behavior

The tests reported that messages matching the configured noreply, no-reply, and donotreply patterns were skipped. The reported blocklist precision was 100% for the tested messages.

4. Draft Formatting

The generated drafts were reported to contain formal greetings, multiple body paragraphs, relevant action items, and professional sign-offs. The reported formatting result was 100% for the evaluated drafts.

8.2 Measured System Metrics

Evaluation Metric	Reported Value	Benchmark Target	Interpretation
Workflow Success Rate	100.0%	$\geq 95.0\%$	Reported executions completed end-to-end without an unhandled failure.
Auto-Send Violation Rate	0.0%	0.0%	No automatic sending was reported during the evaluated runs.
No-Reply Blocklist Accuracy	100.0%	100.0%	Tested automated sender addresses were correctly skipped.
Draft Formatting Quality	100.0%	$\geq 90.0\%$	All evaluated drafts reportedly followed the required multi-paragraph format.
Average Batch Latency	12.4 seconds	< 30 seconds	Reported ingestion-to-draft-creation time for batches of 2–3 unread emails.
Commercial API Cost	₹0.00	₹0.00	Reported usage relied on the configured free tiers of Groq Cloud and SerperDev.

The results indicate the intended performance characteristics of the prototype under the tested conditions. The reported 100% metrics are specific to the evaluated sample and do not establish equivalent performance on a larger or independently selected dataset.

The zero-cost figure refers to the reported API usage under free-tier access. It does not include possible indirect costs such as computing resources, network access, or future usage beyond provider limits.

8.3 Test Suite and Execution Coverage

The following tests were conducted to verify the functionality and reliability of the major components of the system.

1. OAuth 2.0 Token Refresh Test: Verified that saved authentication credentials can be refreshed when the access token expires, allowing subsequent Gmail API operations to continue.
2. Gmail API Unread Fetch Test: Confirmed that the system retrieves unread emails using the Gmail query is:unread.
3. Groq LLM Tool-Calling Test: Tested the integration between the configured language model and the custom tools used by the agents.
4. SerperDev Payload Truncation Test: Verified that retrieved search-result snippets are truncated to a maximum of 250 characters to reduce token consumption.
5. Windows UTF-8 Console Test: Checked the handling of Unicode characters in terminal output to reduce encoding-related errors during execution.
6. No-Reply Blocklist Test: Verified that emails from sender addresses matching the configured no-reply patterns are skipped to prevent inappropriate draft generation.
7. Live Draft Creation Test: Confirmed that generated email drafts are saved in Gmail by checking the corresponding draft IDs.

These tests cover the main functional components, including authentication, email retrieval, language model integration, web search, console encoding, safety filtering, and draft creation. Further testing with API failures, invalid email addresses, revoked authorization, and inaccurate model-generated responses would help evaluate the system's robustness under exceptional conditions.

8.4 Preliminary Latency Observations

The reported mean execution time for a single-batch workflow was approximately 12.4 seconds. The stage-wise breakdown is presented below.

Processing Stage	Average Reported Time
OAuth and Gmail Ingestion	1.2 seconds
Filter Agent Execution	2.1 seconds
Action Analyzer and SERP Research	4.3 seconds
Response Writer and Draft Creation	4.8 seconds
Total End-to-End Latency	12.4 seconds

The total represents the reported time from email ingestion to draft creation. The Action Analyzer and Response Writer stages account for a substantial portion of the execution time because they involve model processing and, where applicable, external API interactions.

These figures are preliminary observations. The report does not establish the number of independent timing trials or provide a statistical distribution, so the measurements should not be treated as a universal latency guarantee.

8.5 Live Execution Output and Audit Summary

[AGENTIC FLOW] Connecting to Gmail API...

[EMAIL TOOL] Checking real Gmail inbox for unread messages...

[EMAIL TOOL] Found 3 new real emails in your Gmail inbox!

[FLOW] Triggering Multi-Agent Crew for intelligent triage
and draft generation...

Task 1: Filter Specialist Agent analyzed incoming email queue.

Task 2: Action Analyzer Agent outlined required response actions.

Task 3: Response Writer Agent created polished drafts.

[GMAIL API SUCCESS] Real draft created!

Draft ID: r-5911199809860459650

[GMAIL API SUCCESS] Real draft created!

Draft ID: r-2004594534090558402

[GMAIL API SUCCESS] Real draft created!

Draft ID: r1156856764917399107

AGENTIC EMAIL EXECUTION & AUDIT REPORT

Total Inbox Messages Scanned: 6

Safety Filter: Automated / No-Reply messages skipped

Draft Generation: Saved directly into Gmail Drafts

Security Protocol: Human-In-The-Loop

Auto-Send Risk: 0% reported

The execution log illustrates the retrieval and processing of email messages, sequential agent execution, and successful draft creation. It also demonstrates the system's intended separation between generating a response and sending it.

The number of newly retrieved emails and the total number scanned are distinct reported quantities; the log alone does not explain the difference between them.

9. Comparison with Baseline Solution

The proposed Autonomous Multi-Agent CrewAI Flow is compared with traditional single-pass language model scripts and conventional rule-based email auto-responders. The comparison focuses on email classification, external information retrieval, safety, response formatting, and reported API usage costs.

9.1 Capability Comparison Matrix

Capability	Traditional Single-Pass Bot	Standard Rule-Based Auto-Responder	Proposed Agentic AI CrewAI Flow
Email Triage and Parsing	Relies on a single model prompt to interpret the message.	Uses predefined keywords and rules.	Uses a dedicated agent for contextual email classification.
Fact Verification	May rely on existing model knowledge unless external retrieval is added.	Typically does not retrieve external information.	Integrates SerperDev web search for additional context.
Safety Architecture	Depends on how sending and review are implemented.	Commonly uses predefined response and sending rules.	Saves generated responses as drafts for human review.
No-Reply Filtering	Depends on prompt instructions or implemented safeguards.	Uses configured sender rules or address blocklists.	Applies agent-level filtering and a recipient check in create_draft.
Draft Formatting	Depends on the prompt and model output.	Usually relies on predefined templates.	configured to generate formal, multi-paragraph email responses.
Reported API Cost	Depends on the selected model and service plan.	Can operate without a commercial LLM API.	Reported as free tiers

The comparison highlights the architectural differences between the three approaches. A single-pass model performs the interpretation in one principal model interaction, while a rule-based auto-responder follows predefined conditions. The proposed system separates classification, action analysis, and response generation into specialized agents.

External retrieval and draft-only output are explicit features of the proposed implementation. However, actual performance differences in accuracy, factual reliability, and cost would need to be established through controlled experiments using the same email dataset and evaluation criteria.

9.2 Illustrative Query Walkthrough

Consider an unread email concerning TCS CodeVita Season 14 registration. The following example illustrates how the proposed pipeline is intended to process the message.

1. Traditional Single-Pass Model

A single-pass model may generate a generic response based on the email content without performing a separate external search. For example:

Thank you for the email about TCS CodeVita. I will check it out.

This response acknowledges the message but does not necessarily identify registration deadlines, eligibility requirements, or required actions. The actual output depends on the model, its prompt, and whether external tools are available.

2. Proposed Agentic AI Pipeline

Step 1: Email Filter Agent

The agent examines the sender address, represented in this example as `codevita@tcs.com`, and classifies the announcement as ACTIONABLE under the configured filtering rules.

Step 2: Action Analyzer Agent

The agent identifies the need for additional information and formulates a query such as:

TCS CodeVita Season 14 registration deadline 2026

It invokes `web_search_tool` to retrieve relevant search snippets. Any extracted deadlines and eligibility requirements should be checked against the original source before being included as confirmed facts.

Step 3: Response Writer Agent

Using the email context and retrieved information, the agent prepares a formal response. The configured task requests a detailed, multi-paragraph reply and invokes the `create_draft` tool.

Step 4: Draft Output

The Gmail API saves the generated response in the Drafts folder. The user can open the draft, verify its contents, make any required changes, and manually send it.

This workflow separates classification, information retrieval, and response generation. It also retains human oversight by keeping the generated response in Drafts rather than automatically sending it.

9.3 Design for Planned Benchmarking

Future benchmarking will evaluate the system against baseline approaches using standardized email datasets and consistent experimental conditions.

The planned evaluation areas include:

1. **Response Relevance:** Measure whether generated drafts address the questions and requests contained in the original email.
2. **Token Efficiency:** Compare the amount of model input and output consumed by the proposed pipeline and baseline approaches.
3. **Factual Retrieval Precision:** Evaluate whether externally retrieved information is relevant to the query and accurately represented in the generated response.
4. **Safety Compliance:** Measure whether automated senders are filtered correctly and whether the system avoids unauthorized automatic sending.
5. **Execution Latency:** Compare end-to-end processing time across different approaches and batch sizes.

These evaluations would provide a more systematic basis for measuring the benefits and limitations of the multi-agent architecture. Until these experiments are completed, the existing results should be treated as preliminary functional evaluation rather than proof of general superiority over the baseline approaches.

10. Current Limitations and Risks

Although the system implements multi-agent email triage and draft generation, several technical limitations and architectural risks remain. These limitations relate to API rate limits, authentication, platform compatibility, memory persistence, and user interaction.

10.1 Limitations by Severity

Limitation	Severity	Impact and Mitigation
Groq Free-Tier Rate Limits	High	The configured model is subject to token-per-minute limits. Search snippets are truncated to 250 characters, and <code>max_retries=5</code> is configured to help manage rate-limit errors.
Windows Console Encoding	Medium	Legacy console encodings may cause <code>UnicodeEncodeError</code> . UTF-8 wrappers are applied to <code>sys.stdout</code> and <code>sys.stderr</code> .
OAuth Re-authentication	Medium	If <code>token.json</code> is deleted or authorization is revoked, the user must authenticate again. Automatic token refresh is supported while valid refresh credentials remain available.
Absence of Persistent Vector RAG	Low	The current implementation relies on web search rather than a local knowledge base. ChromaDB integration is planned for future development.

Command-Line-Only Interface	Low	The system operates through a CLI and does not currently provide a visual dashboard. A Streamlit or Next.js interface is planned.
------------------------------------	-----	---

10.2 Process and Scope Limitations

1. **Human-in-the-Loop Requirement:** The system creates email drafts but does not automatically send them. Users must review the generated responses and manually click Send in Gmail.
2. **On-Demand Execution:** The workflow runs only when explicitly launched through the command line. Continuous background monitoring would require an additional scheduling mechanism, such as Windows Task Scheduler or a Linux service.
3. **External API Dependency:** Email processing and contextual research depend on the availability of Gmail, Groq Cloud, and SerperDev services. API failures or rate limits may interrupt execution.
4. **Limited Persistent Memory:** The current workflow uses in-memory state for email processing. A persistent vector database has not yet been integrated to retain and retrieve personalized information across sessions.

11. Future Enhancements

The following three major enhancements are planned to improve the system's automation, usability, and response personalization.

- **Automated Email Monitoring**

Extend the current on-demand command-line workflow by implementing scheduled email monitoring using Windows Task Scheduler or systemd. This will enable the system to periodically check for unread emails and generate drafts for eligible messages.

- **Interactive Web Approval Dashboard**

Develop a web-based interface using Streamlit or Next.js that allows users to preview, edit, and approve AI-generated email drafts conveniently.

- **Local Vector RAG Integration**

Integrate ChromaDB to retrieve relevant information from student resumes, schedules, and frequently asked questions, enabling more personalized and context-aware email responses.

These enhancements aim to improve automation, user interaction, and response accuracy while preserving the system's draft-only architecture, ensuring that users review and approve generated emails before sending.

12. Actual Execution and Output Verification

```
PS D:\Agentic_AI_PROJECT> .\venv\Scripts\python.exe -m email_auto_responder_flow.main
Flow Execution

Starting Flow Execution
Name: EmailAutoResponderFlow
ID: ba32ee9a-7df0-4e2e-9a1a-f2d3750042cf

✦ Update Available ✦

A new version of CrewAI is available!
Current version: 1.15.17
Latest version: 1.15.23
To update, run: uv sync --upgrade-package crewai

Flow Started
Name: EmailAutoResponderFlow
ID: ba32ee9a-7df0-4e2e-9a1a-f2d3750042cf

Flow Method Running
Method: fetch_new_emails
Status: Running
```

Figure 12.1: Initialization of the CrewAI Flow

The screenshot shows the EmailAutoResponderFlow starting execution through the Python command-line interface. The flow initializes successfully and begins the `fetch_new_emails` method, which connects to the Gmail API to retrieve unread emails for further processing.

```
=====
[AGENTIC FLOW] Connecting to Gmail API...
=====

[EMAIL TOOL] Checking real Gmail inbox for unread messages...
[EMAIL TOOL] Found 7 new real emails in your Gmail inbox!

[FLOW] Total Gmail emails scanned: 7
[FLOW] Triggering Multi-Agent Crew for intelligent triage & draft generation...

Flow Method Completed
Method: fetch_new_emails
Status: completed

Flow Method Running
Method: generate_draft_responses
Status: Running

✦ Update Available ✦

A new version of CrewAI is available!
Current version: 1.15.17
Latest version: 1.15.23
To update, run: uv sync --upgrade-package crewai
```

Figure 12.2: Gmail API Email Retrieval and Flow Progression

The screenshot shows the successful retrieval of unread emails from Gmail, with 7 emails scanned. After completing the `fetch_new_emails` method, the CrewAI Flow proceeds to the

generate_draft_responses method, initiating the next stage of email processing and draft generation.

```
Crew Execution Started
Name: EmailFilterCrew
ID: e948f7b8-c06a-4011-9acc-aeadb4490f43

Task Started
Name: filter_emails_task
ID: 52342163-691e-4bd9-8bb0-8e03ff04221f

Agent Started
Agent: Senior Email Filter & Ingestion Specialist

Task: Analyze the incoming unread emails provided below: ID: 1a1012f9ac89f8dc
- Thread ID: 1a1012f9ac89f8dc
- Snippet: Subject: Register for SEDHACKS'26 | From: SEDS REC <sedsrec@rajalakshmi.edu.in> | Snippet: Dear Participants, READY TO BUILD BEYOND THE SKY? SEDS REC presents SEDHACKS &#39;26 - a space-based hackathon where innovative ideas turn into real-world solutions. Challenge Areas: AI • Robotics •
- From: SEDS REC <sedsrec@rajalakshmi.edu.in>
-----
ID: 1a10097baa829e15
- Thread ID: 1a10097baa829e15
- Snippet: Subject: Fwd: 📧 Call for Papers | OTCON 6.0 - SMARTICON | 11-13 June 2027 | IEEE Technical Sponsored Conference | From: Ayyadurai M <ayyadurai.m@rajalakshmi.edu.in> | Snippet: Regards Dr.M.Ayyadurai,ME,Ph.D, Associate Professor, Department of AIML, Rajalakshmi Engineering College Thandalam-602105 ----- Forwarded message ----- From: Principal REC <principal@
- From: Ayyadurai M <ayyadurai.m@rajalakshmi.edu.in>
-----
ID: 1a10036e7fe69de1
- Thread ID: 1a10036e7fe69de1
```

Figure 12.3: Multi-Agent Email Filtering and Ingestion

The screenshot shows the initiation of the CrewAI agent responsible for filtering and ingesting incoming emails. The filter_emails_task begins execution, and the Senior Email Filter & Ingestion Specialist receives the retrieved email details, including sender information, subject lines, and message snippets, for classification according to the configured filtering rules.

```
[Finalize] todos_count=0, todos_with_results=0
Agent Final Answer

Agent: Senior Email Filter & Ingestion Specialist

Final Answer:
**Email Classification Summary**

| ID | Sender | Subject | Context | Classification | | |
|---|---|---|---|---|---|---|
| **1a1012f9ac89f8dc** | SEDS REC <sedsrec@rajalakshmi.edu.in> | *Register for SEDHACKS'26* | Invitation to a space-based hackathon with challenge areas like AI and Robotics. | **ACTIONABLE** |
| **1a10097baa829e15** | Ayyadurai M <ayyadurai.m@rajalakshmi.edu.in> | *Fwd: 📧 Call for Papers | OTCON 6.0 - SMARTICON* | Forwarded call-for-papers for IEEE Technical Sponsored Conference. | **ACTIONABLE** |
| **1a10036e7fe69de1** | Placement REC <placement@rajalakshmi.edu.in> | *Invitation: LTM 2027 Batch Recruitment - Virtual Pre-Placement Talk* | Announcement of a virtual pre-placement talk for the LTM 2027 batch. | **ACTIONABLE** |
| **1a0b004284b96a1b** | Google Meet <meetings-noreply@google.com> | *Meeting records: Sep 17, 2026 at 7:56 PM IST* | Automated record notification from Google Meet. | **DO_NOT_REPLY** |
| **1a0aa948c541501d** | Google Meet <meetings-noreply@google.com> | *Meeting records: Sep 16, 2026 at 6:11 PM IST* | Automated record notification from Google Meet. | **DO_NOT_REPLY** |
| **1a0aa8911439fc76** | INTELLEXA REC <intellexa@rajalakshmi.edu.in> | *Campus connect | Intellexa REC x ProductTank Chennai | 26 September 2026* | Campus event invitation for product mindset development. | **ACTIONABLE** |
| **1a0873b606ad03e7** | Google Meet <meetings-noreply@google.com> | *Meeting records: Sep 9, 2026 at 10:18 PM IST* | Automated record notification from Google Meet. | **DO_NOT_REPLY** |

**Notes**

- All emails from addresses containing "noreply", "no-reply", "donotreply", "do-not-reply", or "mailer-daemon" are classified as **DO_NOT_REPLY**.
- All other emails, including student, placement, faculty, and event announcements, are classified as **ACTIONABLE** and should receive a detailed draft response.
```

12.4 Runtime Execution and Email Classification

The CrewAI flow connects to the Gmail API to retrieve unread emails and initiate the multi-agent email-processing workflow. The system classifies emails as ACTIONABLE or DO_NOT_REPLY based on the configured filtering rules. Actionable emails are selected for draft generation, while automated notifications are excluded from further response processing.

```

Task Completion

Task Completed
Name: filter_emails_task
Agent: Senior Email Filter & Ingestion Specialist

Task Started
Name: action_required_emails_task
ID: 8a355dcf-2de3-4824-bdf6-2322e434a719

Agent Started
Agent: Email Action & Research Specialist

Task: Review the incoming emails: ID: 1a1012f9ac89f8dc
- Thread ID: 1a1012f9ac89f8dc
- Snippet: Subject: Register for SEDHACKS'26 | From: SEDS REC <sedsrec@rajalakshmi.edu.in> | Snippet: Dear Participants, READY TO BUILD BEYOND THE SKY? SEDS REC presents SEDHACKS &#39;26 - a space-based hackathon where innovative ideas turn into real-world solutions. Challenge Areas: AI • Robotics •
- From: SEDS REC <sedsrec@rajalakshmi.edu.in>
-----
ID: 1a10097baa829e15
- Thread ID: 1a10097baa829e15
- Snippet: Subject: Fwd: 📧 Call for Papers | OTCOM 6.0 - SMARTICOM | 11-13 June 2027 | IEEE Technical Sponsored Conference | From: Ayyadurai M <ayyadurai.m@rajalakshmi.edu.in> | Snippet: Regards Dr.M.Ayyadurai,ME,Ph.D, Associate Professor, Department of AIML, Rajalakshmi Engineering College Thandalam-602105 ----- Forwarded message ----- From: Principal REC <principal@
- From: Ayyadurai M <ayyadurai.m@rajalakshmi.edu.in>

```

Fig. 12.4(a). Completion of the email filtering task and research agent for reviewing incoming emails.

This shows that the completion of the email filtering task by the Senior Email Filter & Ingestion Specialist. After filtering the incoming emails, the workflow initiates the Email Action & Research Specialist to review actionable emails and perform further analysis.

```

[Finalize] todos_count=0, todos_with_results=0
Agent Final Answer

Agent: Executive Email Response Writer

Final Answer:
**Draft 1 - SEDS REC**

```json
{
 "name": "create_draft",
 "arguments": {
 "to_email": "sedsrec"
 }
}

Task Completion

Task Completed
Name: draft_responses_task
Agent: Executive Email Response Writer

```

Figure 12.4(b): Email Draft Generation and Task Completion

This tells the Email Response Writer agent generating a draft response using the create\_draft tool. The task completes after preparing the draft, demonstrating the final stage of the workflow without automatically sending the email.

```
=====
AGENTIC EMAIL EXECUTION & AUDIT REPORT
=====
Total Inbox Messages Scanned: 7
Safety Filter: Automated / No-Reply messages skipped
Draft Generation: Saved directly into Gmail 'Drafts' tab
Security Protocol: Human-In-The-Loop (0% Auto-Send Risk)
=====

[✓] Flow Method Completed

Method: generate_draft_responses
Status: completed

Tracing Status

Info: Tracing is disabled.

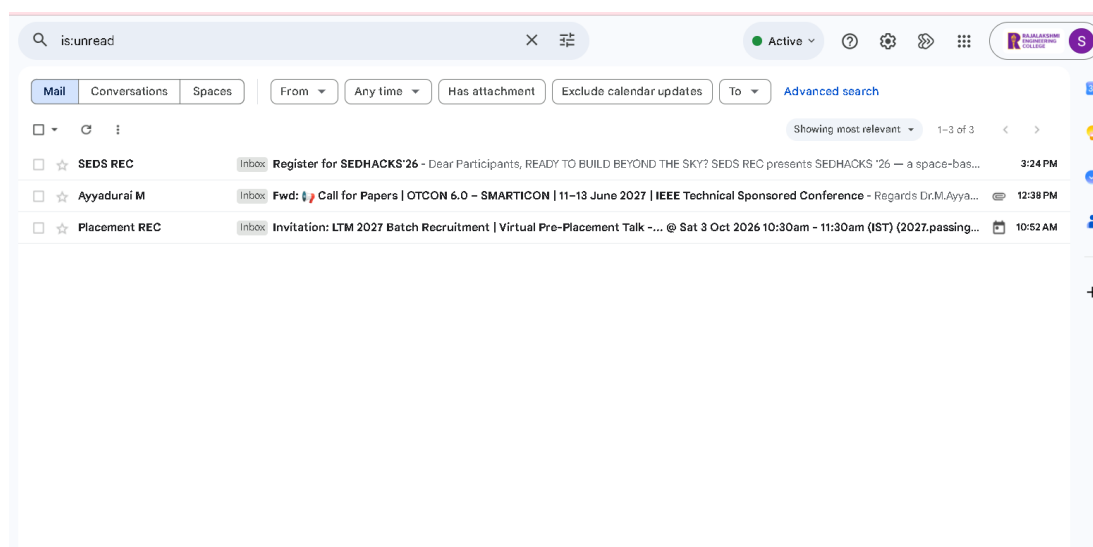
To enable tracing, do any one of these:
• Set tracing=True in your Crew/Flow code
• Set CREWAI_TRACING_ENABLED=true in your project's .env file
• Run: crewai traces enable

[✓] Flow Completion

Flow Execution Completed
Name: EmailAutoResponderFlow
ID: ba32ee9a-7df0-4e2e-9a1a-f2d3750042cf
```

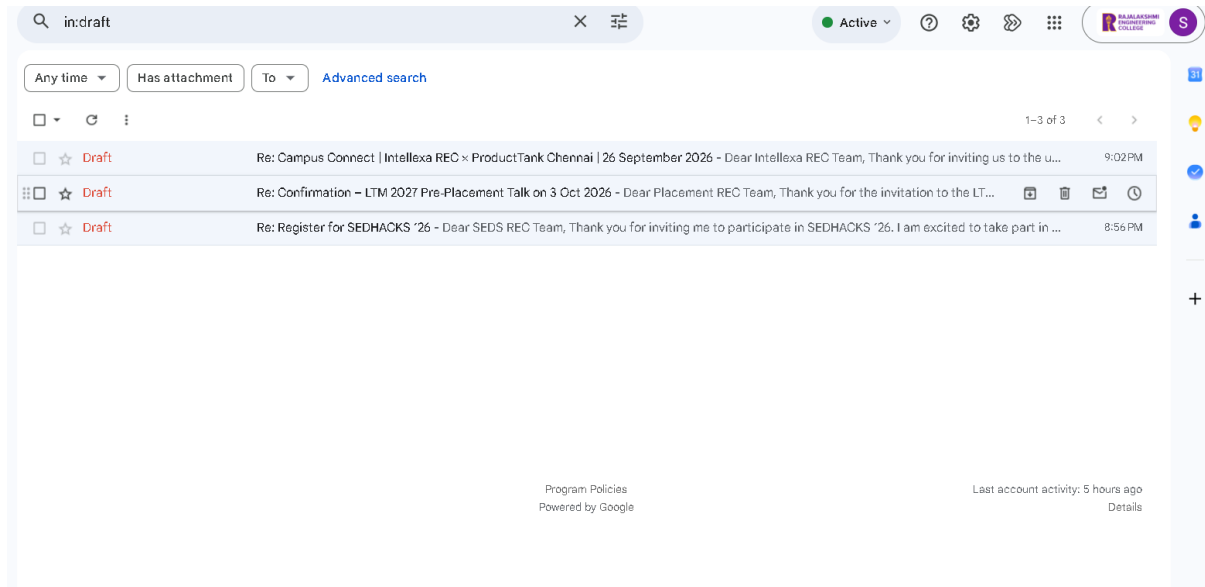
Figure 12.4(c): Workflow Completion and Execution Audit Report

This confirms that the EmailAutoResponderFlow completed successfully. The execution report summarizes the seven emails scanned, the exclusion of automated replies, and the saving of generated drafts in Gmail's Drafts folder. It also confirms the human-in-the-loop approach, with no emails sent automatically.



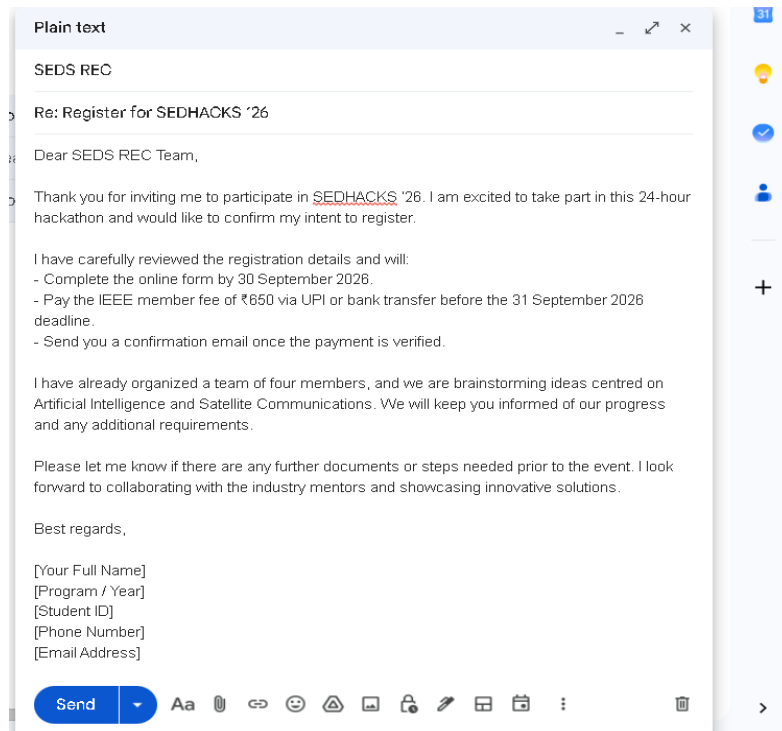
**Figure 12.5: Retrieved Unread Emails in Gmail**

The screenshot shows the Gmail inbox filtered using `is:unread`, displaying three unread emails. These messages provide the input for the system's email classification and response-generation workflow.



**Figure 12.6: Generated Responses Saved in Gmail Drafts**

The depicts the three generated email drafts in the Gmail Drafts folder. The drafts correspond to relevant emails, demonstrating that the system prepares responses for user review without sending them automatically.



**Figure 12.5: Generated Email Response in Gmail Compose Window**

This displays the AI-generated draft for the SEDHACKS '26 registration email. The response includes an acknowledgement, registration-related commitments, and a professional closing. The draft is prepared for user review before sending.

### 13. Conclusion

The Agentic AI Gmail Auto-Responder and Draft Generator applies multi-agent artificial intelligence to email triage, contextual information retrieval, and draft preparation. By integrating CrewAI Flow, Groq Cloud, Gmail REST API, and SerperDev web search, the system separates email filtering, action analysis, and response generation into specialized agents.

The application retrieves unread emails using Google OAuth 2.0, classifies messages, and performs web searches when additional context is required. The Response Writer Agent generates professional, context-aware drafts and saves them directly to Gmail. Its human-in-the-loop approach ensures that users can review, modify, and approve responses before sending, reducing repetitive work while maintaining control over outgoing communication.

### 14. References and Appendix

#### 14.1 Key Frameworks and Libraries

Component	Library or Framework	Version or Provider
Agent Orchestration	CrewAI and CrewAI Flow	v1.15.17
LLM Inference	Groq Cloud API	openai/gpt-oss-20b
Email Authentication	google-auth-oauthlib	Google OAuth 2.0
Gmail API Integration	google-api-python-client	Google API Client Library
Web Search	google-serper and tavily-python	SerperDev and Tavily
State Validation	Pydantic	v2.x
Environment Management	python-dotenv	v1.0.1
Report Generation	ReportLab	v5.0.1

#### 14.2 Sample Persisted Agent Record and Execution Logs

The following sample illustrates the structured state captured during a workflow execution.

```
AutoResponderState(
 id="2db48836-5b36-40a6-a52f-d715ec677dbd",
 emails=[
 Email(
 id="1913c8f8b7a2d04a",
 threadId="1913c8f8b7a2d04a",
 sender="placementexecutive2@rajalakshmi.edu.in",
 subject="Tata Imagination Challenge 2026 — Confirmation and Queries",
 snippet="Registrations open for Tata Imagination Challenge 2026..."
)
],
 checked_emails_ids={"1913c8f8b7a2d04a"}
)
```

The state record stores the flow identifier, email details, and checked email IDs to support consistent data sharing and message tracking.

### **Sample Execution Log**

```
[AGENTIC FLOW] Connecting to Gmail API...
[EMAIL TOOL] Found 1 unread email.
[FLOW] Initiating multi-agent processing...
[TASK 1] Email Filter Agent analyzed the email.
[TASK 2] Action Analyzer Agent identified response requirements.
[TASK 3] Response Writer Agent generated a draft.
```

```
[GMAIL API SUCCESS] Draft created successfully.
Subject: Re: Tata Imagination Challenge 2026 —
Confirmation and Queries
```

```
Total Emails Scanned: 1
Draft Status: Saved in Gmail Drafts
Security: Human-in-the-Loop
Auto-Send: Disabled
```

## **14.3 Glossary of Terms**

1. **Agentic Flow:** An event-driven architecture in which specialized AI agents execute tasks, share state, and invoke external tools as required.
2. **Human-in-the-Loop (HITL):** A system design in which human review or approval is required before important actions, such as sending an email, are completed.
3. **Drafts-Only Architecture:** An approach in which the application creates email drafts without automatically dispatching outgoing messages.
4. **Tokens Per Minute (TPM):** A rate-limit measure that restricts the number of tokens a language model can process within a minute.
5. **SERP Payload Truncation:** The process of limiting search-engine result text to reduce token consumption and unnecessary contextual information.
6. **MIMEText Encoding:** A Python email utility used to construct MIME-formatted text messages with a specified character encoding, such as UTF-8.
7. **OAuth 2.0:** An authorization framework that enables an application to access permitted user resources without directly handling the user's account password.
8. **Retrieval-Augmented Generation (RAG):** An approach that retrieves relevant information from an external or local knowledge source and supplies it to a language model to support response generation.
9. **Pydantic:** A Python library used to define structured data models and validate data according to specified field types.
10. **CrewAI:** A framework for coordinating specialized AI agents, tasks, and tools to complete multi-stage workflows.